SYNTHAGENT: A System for Reliable LLM-Based Synthetic Data Generation in Small Data

Sancho Amaral Simões[1,*], João P. Vilela[1,2], Pedro Henriques Abreu[1]
[1]*CISUC-DEI - FCTUC, University of Coimbra*, Coimbra, Portugal
[2]*CRACS/INESCTEC-DCS - FCUP, University of Porto*, Porto, Portugal
[*]Corresponding author
sanchosimoes@student.dei.uc.pt, jvilela@fc.up.pt, pha@dei.uc.pt

***Abstract*—A dataset is small for a learning task when it holds fewer effective samples than the task requires, a common situation in healthcare. Synthetic data generation (SDG) can enlarge such datasets, but a generator fitted only to the scarce samples, such as SMOTE, a generative adversarial network or a variational autoencoder, cannot add information that those samples lack. Large language models (LLMs) can: they are not trained on the target data and generate new samples from a few real examples and the knowledge acquired during pre-training. Their output, however, is unreliable: a response can contain fewer samples than requested, malformed records, missing or invented features, or values of the wrong type, and its cost is rarely reported. This paper presents SYNTHAGENT, a system that places a control layer between the LLM and the generated dataset. The control layer selects the real examples shown to the model, validates every response against the dataset schema, corrects invalid records, requests missing samples until the requested size is reached, and records the cost of each step. We evaluate SYNTHAGENT on six public medical classification datasets with 195 to 768 samples and compare it with SMOTE, CTGAN and TVAE. Without any training on the target data, SYNTHAGENT completes every run with 99.7% to 100% of the requested values filled. With GPT-4.1 as the LLM, it obtains the largest average F1 gain of the five generators and never lowers the average F1 of a dataset by more than 0.6 points, whereas SMOTE and CTGAN lose up to 8.2 and 10.6 points on individual datasets.**

***Index Terms*—small data, synthetic data generation, tabular data, large language models, data augmentation, healthcare**

I. INTRODUCTION

Context: small data: Supervised learning assumes that the training data contain enough information to estimate a model for the target population. In healthcare this assumption often fails: a dataset is limited by the number of patients a hospital can recruit, by the prevalence of the disease, and by privacy rules that prevent pooling data across institutions [1]. Whether a dataset is too small, however, depends on the task: 300 patients can be enough to predict a common outcome from a few features and far too few for a rare outcome or many features. We therefore use the task-relative definition of our recent survey [1], which combines learning-theoretic sample complexity with the sample-size criteria used for clinical prediction models [2].

Definition 1 (Small data [1]). Let $n_T^\star$ be the *sample complexity* of a task T, the smallest number of independent, correctly labelled samples with which a model reaches a required error with high probability. Let $n_{\text{eff}}(D)$ be the *effective sample size* of a dataset D, the number of such samples that would carry the same information as D once clustering, missing labels, label noise and population shift are accounted for. The *smallness index* is $\sigma_T(D) = n_T^\star / n_{\text{eff}}(D)$, and D is *small* for T when $\sigma_T(D) > 1$. The regimes $1 < \sigma \leq 10$, $10 < \sigma \leq 100$ and $\sigma > 100$ are called *strained*, *small* and *extreme*.

Small data is therefore a property of the whole dataset relative to the task, not of one class. Class imbalance enters the definition only through $n_T^\star$, which for a binary outcome is driven by the number of events rather than the number of patients [2]. The six medical datasets in our study have 195 to 768 samples; estimated with the criterion of Riley et al., their smallness index ranges from 1.0 (diabetes, at the boundary) to 42.1 (five-class heart disease), so all are small in the sense of Definition 1 (Table III).

Strategies for small data: Our survey classifies remedies for small data by where the missing information comes from [1]: data from other institutions (pooling), related tasks or pre-trained models (proxy), domain knowledge (prior), unlabelled data, new data collection, and the transformation of existing data, which includes synthetic data generation (SDG). For SDG it proves a conservation result (Proposition 4.1 in [1]): a synthetic dataset produced from D with the help of an external resource M carries no more information about the target than D and M together. A generator fitted only to D, such as SMOTE, CTGAN or TVAE, has no M; it can still help a learner use D more efficiently, or provide data for software testing, but it cannot add knowledge that D lacks. A pre-trained generator is different: it brings the knowledge acquired on other data. This is the reason to use large language models (LLMs) for SDG in small data. An LLM is not trained on D; it receives a few real samples in the prompt and generates new ones using what it learned during pre-training, for example that a feature named `age` is a positive integer that increases with blood pressure [3], [4].

Problem: The difficulty is that LLM output is not reliable. A response may contain fewer samples than requested, malformed JSON, missing or invented features, or values of

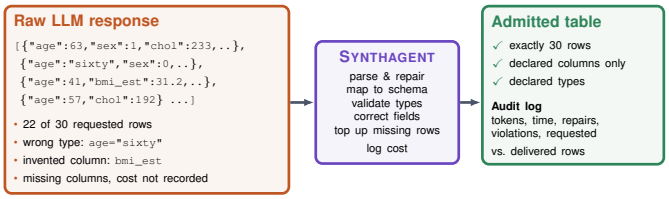


Fig. 1. The problem addressed in this paper. Asked for 30 samples of a heart-disease dataset, an LLM response (left) can contain too few samples, invented or missing features and values of the wrong type. SYNTHAGENT (right) validates and corrects every response, so the resulting dataset has the requested size and schema, and a log records each correction and its cost.

TABLE I
REQUIREMENTS MET BY EXISTING GENERATOR FAMILIES AND BY SYNTHAGENT. ✓: BY DESIGN; (✓): PARTIALLY (SMOTE'S SIZE FOLLOWS THE CLASS BALANCE; FINE-TUNED LMS DISCARD UNPARSABLE SAMPLES); ×: NOT ADDRESSED.

Family	No training on D	R1: exact size	R2: schema validated	R3: cost measured
Interpolation [5]	✓	(✓)	×	×
Deep generators [6]–[8]	×	✓	×	×
Fine-tuned LMs [3], [9]	×	✓	(✓)	×
Prompted LLMs [4], [10]	✓	×	×	×
SYNTHAGENT (ours)	✓	✓	✓	✓

the wrong type (Figure 1, left), and every request has a cost in tokens and time. Without a mechanism that detects and corrects these failures, the output of an LLM cannot be used as a dataset. Generators fitted to D have the opposite profile: their output always has the declared features and size, but in small data they estimate the distribution from the same few samples; in our experiments, only 29.6% of the samples produced by CTGAN respect the value ranges and types of the real data (Section VI-D).

Problem statement. *Given a small dataset D (Definition 1) with schema S and a requested number of synthetic samples N, generate a synthetic dataset G such that (R1) G has exactly N samples, (R2) every sample conforms to S, and (R3) the cost of generation is measured, without training a generator on D; and such that G is (R4) faithful to D, (R5) does not reproduce the samples of D, and (R6) improves, or at least does not degrade, classifiers trained on D.* Requirements R1–R3 are properties a system can guarantee by construction; R4–R6 can only be measured.

Existing approaches and their limitations: Table I groups existing generators by the requirements they meet. Interpolation methods such as SMOTE [5] need no training, but they only create samples between existing minority-class samples, add no external information and do not respect feature types. Deep generators [6]–[8] must be trained on D, which small data does not support, and language models fine-tuned on D [3], [9] bring pre-trained knowledge but still require training on the scarce samples. Prompted LLMs [4], [10] need no training and have been proposed for the low-data setting, but they study what to include in the prompt and which generated samples to keep; they do not guarantee R1–R3. Finally, generators are usually compared from single runs. In small data this is not enough: two runs of the same generator can differ as much as two different generators (Section VI-A).

Our approach: We propose SYNTHAGENT, a system that separates two tasks: the LLM *proposes* samples, and a deterministic *control layer* decides which of them enter the synthetic dataset. The control layer is the part of the system that surrounds every LLM request. It keeps a record of the schema, the real samples shown to the model, the samples accepted so far and the number still missing, and it has four components, each of which removes one of the failures in Figure 1. A *context selector* chooses which real samples fit in the prompt. A *parser* repairs malformed output and maps the returned features to the schema. A *validator* checks feature types and asks the model to correct only the invalid values. A *completion loop* requests exactly the missing samples until N is reached. Every token, correction and violation is logged. With these components, SYNTHAGENT meets R1–R3 without training on D, and the experiments measure R4–R6.

Research questions: The objective of this paper is to determine whether a control layer makes LLM-based SDG a reliable and useful option for small data. We answer it through four research questions, each building on the previous one:

RQ1 *Reliability.* Does SYNTHAGENT deliver datasets that meet R1–R3, and which components of the control layer change the result beyond the variation between repeated runs?

RQ2 *Quality.* Are the generated samples faithful to the real data (R4) without reproducing them (R5)?

RQ3 *Utility.* Do the generated samples improve classifiers trained on small data (R6), given how much those classifiers already vary between data splits?

RQ4 *Comparison.* How does SYNTHAGENT compare with established generators in quality, utility and cost, and how much does the result depend on the LLM?

Contributions:

1) SYNTHAGENT, an open-source system that makes LLM-based SDG for small data meet R1–R3 by construction (Section IV).
2) An evaluation protocol for SDG in small data that pairs each classifier trained with synthetic samples with the same classifier trained on real samples only, on the same data split, and that estimates the variation between repeated runs before comparing configurations (Section V).
3) An empirical study on six medical datasets with four LLMs and three established generators, 780 dataset–configuration runs and up to 23,400 classifier fits (Section VI).

Main findings: The study supports three claims. First, the control layer is sufficient to make LLM output usable as a dataset: with each of the four LLMs analysed, SYNTHAGENT completed all six datasets and ten runs with 99.7% to 100% of the values filled, and the type violations of the fastest LLM were detected and sent back for correction instead of entering the dataset (RQ1). Second, with SYNTHAGENT, LLM-based

SDG is the most robust augmentation in F1 among the five generators: with GPT-4.1 it gives the largest average F1 gain ($+1.09$ points) and never lowers the average F1 of a dataset by more than 0.6 points, while SMOTE, CTGAN and TVAE lose up to 8.2, 10.6 and 2.9 points (RQ3, RQ4). Third, in small data the variation between repeated runs is as large as the effect of most design choices; only the choice of LLM exceeds it, and no generator improves utility significantly on all six datasets (RQ1, RQ4).

II. Background and Related Work

A. Synthetic Data Generation for Tabular Data

Tabular SDG methods fall into four families. *Interpolation and statistical models* include SMOTE, which creates minority-class samples on the segment between two neighbours [5], and Bayesian networks, also used for differentially private release [11]. *Deep generative models* learn the joint distribution with a neural network: CTGAN and TVAE [6], diffusion models such as TabDDPM, CoDi and TabSyn [7], [8], [12], and relational or cross-silo variants [13], [14]. *Fine-tuned language models* serialize each sample as text and fine-tune a language model on the target dataset: GReaT [3], REaLTabFormer [9] and TabLoft [15]. *Prompted LLMs* generate samples from examples placed in the prompt, without training: CLLM [4] and EPIC [10].

B. Synthetic Data Generation in Small Data

For small data, the four families have the following limitations.

- L1 *No external information.* Interpolation and deep generators are fitted only to D, so by the conservation result of [1] they cannot add information about the target; they also ignore what the features mean.
- L2 *Training on scarce samples.* Deep generators and fine-tuned language models estimate many parameters from the few samples available, so they can produce samples outside the observed ranges and types, or reproduce training samples [16].
- L3 *Unreliable output.* Prompted LLMs need no training and use feature semantics, but nothing ensures that their output is a valid dataset of the requested size, and their cost is not measured. CLLM [4] filters generated samples by their learning dynamics and EPIC [10] designs balanced prompts; both focus on the quality of the samples that are kept, not on R1–R3.
- L4 *Evaluation without a noise estimate.* Generators are compared from single runs, so differences caused by run-to-run variation, which is large in small data, are reported as effects.

SYNTHAGENT addresses L3 with its control layer and L4 with its evaluation protocol. Because it uses a pre-trained LLM, it also avoids L1 and L2.

C. Validated Output and Data Quality

Data-management systems have long combined generation or cleaning with integrity constraints. Kamino enforces denial constraints during private synthesis [17], and data-cleaning systems combine statistical inference with constraints [18], [19]. Foundation models can clean and integrate data through prompting [20], and recovering structured records from free-form LLM output is a data-engineering problem in its own right [21]. SYNTHAGENT applies the same principle to SDG: the schema and type rules are inferred from the data, and every LLM response is parsed and validated against them. Studies of data preparation show that a step that looks beneficial can harm downstream models [22], [23]; we therefore evaluate synthetic samples with paired downstream models.

D. Evaluating Synthetic Data

Synthcity and SynthEval recommend evaluating fidelity, privacy and utility together [24], [25]. A classifier two-sample test measures whether a classifier can distinguish real from synthetic samples [26]. Membership-inference attacks measure whether an attacker can tell which samples were used for generation [16], [27]. Formal guarantees require differential privacy [28], which SYNTHAGENT does not provide; we use a membership-inference attack as a diagnostic only.

III. Problem Formulation

We now formalize the problem statement of Section I. Let D^{tr} and D^{te} be a training and a test partition of a small dataset D with schema S. A configuration θ of the system (LLM, context strategy, active rules) produces a synthetic dataset G_θ from D^{tr} without access to D^{te}. Requirements R1 and R2 define the feasible set $\mathcal{F}(S, R, N)$ of datasets with exactly N samples that conform to S and to the active rules R. The remaining requirements are objectives: a fidelity loss $L_{\mathrm{fid}}(D^{\mathrm{tr}}, G_\theta)$ for R4, a privacy risk $L_{\mathrm{priv}}(D^{\mathrm{tr}}, G_\theta)$ for R5, the change in downstream utility $\Delta U(G_\theta)$ for R6 and the generation cost $\kappa(\theta)$ for R3:

$$\theta^\star \in \operatorname*{arg\,min}_{\theta:\, G_\theta \in \mathcal{F}(S,R,N)} \big(L_{\mathrm{fid}},\ L_{\mathrm{priv}},\ -\Delta U,\ \kappa\big)(\theta). \qquad (1)$$

Equation (1) has several objectives because the requirements of Section I can conflict: samples that are closer to the real ones are more faithful but easier to attack, and a larger LLM may give better samples at a higher cost. We do not combine the objectives into one weighted score, because the right weights depend on the use: a hospital sharing data for software testing weights privacy and validity, while a team augmenting a classifier weights utility. SYNTHAGENT guarantees the constraint $G_\theta \in \mathcal{F}(S, R, N)$ (Section IV); Section V defines how each objective is measured.

IV. The SynthAgent System

This section describes how the control layer guarantees R1–R3. Figure 2 shows the architecture. A profiling step infers the schema S and the type rules R from the training data. Each generation request then goes through four stages: the context

selector builds the prompt (1), the LLM proposes a batch of samples (2), the parser and validator check it (3), and the accepted samples are counted against the requested size (4). Each type of failure triggers its own correction.

A. State of the Control Layer

Let D^{tr} be the training data with schema S, N the requested number of synthetic samples and R the active rules. At step t, the control layer keeps the state

$$z_t = (S,\, R,\, C_t,\, A_t,\, N - |A_t|,\, B_t,\, E_t), \tag{2}$$

where C_t is the set of real samples shown to the LLM, A_t the accepted synthetic samples, $N-|A_t|$ the number still missing, B_t the remaining token budget and E_t the log of parser and validator results. The LLM, denoted π_ϕ, proposes a batch $Y_t \sim \pi_\phi(\cdot \mid z_t)$. Let P_S be the parser, which also maps the returned features to S, $V_{S,R}$ the validator and K a correction step that sends the invalid values back to the LLM, at most twice. The accepted set is updated as

$$A_{t+1} = A_t \cup K_{V_{S,R}}\big(P_S(Y_t)\big). \tag{3}$$

Equation (3) expresses the main design decision: the LLM only proposes, and a sample enters A_{t+1} only after it has been parsed, mapped to S and validated. The state is explicit rather than implied by the conversation history, so the number of missing samples is always known, and E_t records every failure for later inspection.

B. Context Selection (R1, R3)

The context selector decides which real samples the LLM sees. Table II lists exactly what the model receives in our experiments. The core evidence is a 20% sample of the real training split, serialized as compact row-major JSON with the ordered feature columns and the observed target, so the model sees how features and class co-occur in real samples. No free-text description of the clinical task is added, so that no dataset receives more prompt engineering than another; the model must infer the role of each feature from its name and values.

Formally, a selection strategy $s \in \{\mathrm{random}, \mathrm{dense}, \mathrm{diverse}\}$ defines an ordered candidate pool, and the token budget determines how much of that pool is visible:

$$\begin{aligned} C_t &= \mathrm{Slice}_t\big(\mathrm{Select}_s(D^{\mathrm{tr}}, \lceil 0.2|D^{\mathrm{tr}}|\rceil), k_t\big)\,, \\ k_t &= \max\{k : T(H, I, C_{t,k}, O_t) \le 0.85W\}. \end{aligned} \tag{4}$$

Here H is the schema and output contract, I optional instruction text, O_t the reserved output, W the backend context window and T a token estimator (2.8 characters per token for compact JSON). The slice operator rotates through the pool across chunks, so successive requests see different patients when the pool exceeds one window. If the provider still rejects a request as too long (HTTP 400 or 422), the control layer shrinks the visible context by 40% and retries, at most three times. "20% context" therefore describes the candidate pool, not a promise that every candidate row appears in every prompt.

The 20% fraction and the 85% window limit were fixed before the experiments and not tuned. The default strategy samples the context at random. *Dense* prefers rows with the most observed cells, reducing ambiguity from missing fields. *Diverse* standardizes numerical features, runs k-means and selects the rows nearest to the centroids, so that the context covers more of the feature space. Neither strategy is class-stratified, so neither guarantees that a rare diagnosis appears in context; Section VII returns to this limitation.

C. Parsing, Validation and Correction (R2)

The parser and the validator decide which parts of a response can enter the accepted set A_t of Equation (3). Samples are requested in JSON batches of at most 30, a size that keeps every response within the output limit of the smallest LLM tested and limits the loss when a response fails. A local parser first extracts structured content and fixes common defects (non-finite literals, trailing commas, concatenated objects). If parsing still fails, the control layer issues one repair request at temperature 0.1 that quotes the error position and surrounding text. The returned feature names are then mapped back to the schema.

Profiling infers a type rule for every feature, and optional boundary rules (numerical ranges with 10% tolerance, or categorical domains with at most 20 values). Dynamic Pydantic models validate each batch. The validator groups failures by field and reports one representative bad value per field, so the correction prompt does not grow linearly with repeated errors. Up to two correction rounds are attempted per batch. User-defined post-processing rules (noise, rounding, normalization) can be applied deterministically afterwards.

In the reference configuration of our experiments, type rules are active, boundary rules are off and no user rules are supplied, so we evaluate the default system rather than the strictest one. Rules are used only by the validator and are never inserted into the prompt.

D. Completion Loop (R1)

LLMs often return fewer samples than requested. The completion loop derives its next request from the accepted state rather than from the conversation:

$$r_t = \max(0, N - |A_t|), \qquad q_t = \min(30, r_t). \tag{5}$$

Equation (5) requests only the residual q_t, and any overshoot is truncated. The loop runs for at most 50 top-up rounds and stops after five consecutive empty or unparsable responses. Successful runs therefore return exactly N rows; persistent provider failure can still end a run short, and that shortfall is recorded rather than hidden. Algorithm 1 summarizes the complete loop.

Audit log and cost accounting (R3): Every run records prompt, completion and total tokens, LLM generation time, parser repair attempts, rule violations, the requested and delivered row counts and the model identifier. These records make a synthetic dataset inspectable after the fact and give the cost term $\kappa(\theta)$ in Equation (1) a measured value.

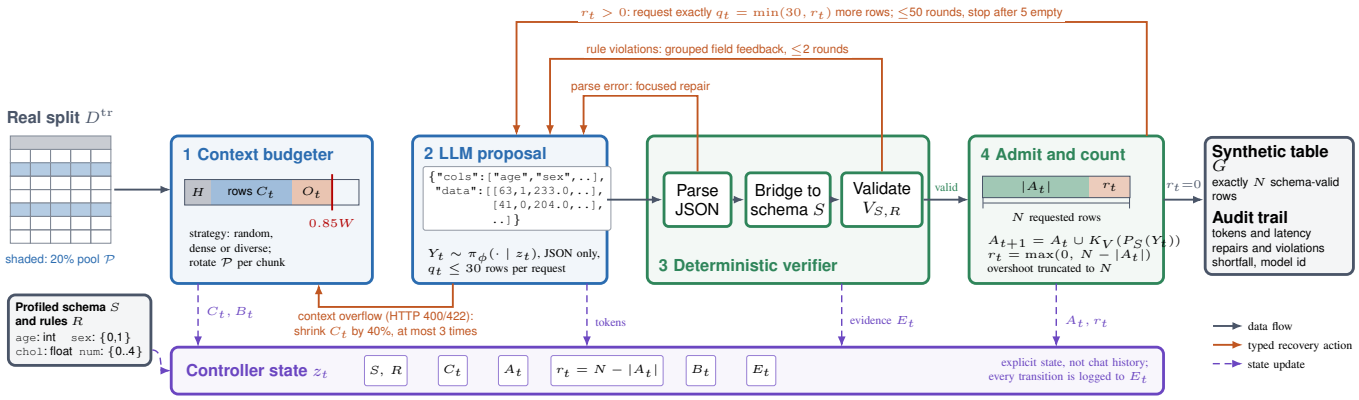


Fig. 2. Architecture of SYNTHAGENT. Real samples enter a context window of limited size (1); the LLM proposes a JSON batch (2); deterministic checks parse it, map it to the schema and validate the type rules (3); and accepted samples are counted against the requested size (4). Each failure type triggers its own correction (orange). Every stage reads or updates the state z_t of the control layer (purple), which also serves as the audit log of the run.

TABLE II
INFORMATION BOUNDARY FOR ONE GENERATION REQUEST IN THE REPORTED STUDY. "NOT SUPPLIED" IS INTENTIONAL: THOSE SIGNALS CANNOT INFLUENCE THE GENERATED ROWS.

Context layer	What the model receives	Why it is included or excluded
Schema	Ordered column names and an exact column-count requirement	Fixes the record interface and prevents invented or omitted fields.
Demonstrations	Compact row-major serialization of a 20% sample of the training split, with feature and target values	Exposes joint patterns, value formats, label prevalence and plausible cross-column combinations without sending the whole local table.
Generation task	Exact number of new rows for the current chunk, at most 30	Turns an open-ended completion into a bounded action whose output count can be verified.
Output contract	JSON only, fixed column order, complete cells, raw numeric values, no extra metadata	Makes parsing deterministic and removes interface failures unrelated to statistical quality.
Dynamic feedback	Parser error location after malformed JSON, or grouped field violations after failed validation	Reveals only the evidence needed for a targeted repair instead of replaying the full history.
Free-text metadata	Not supplied	Prevents unequal, dataset-specific prompt engineering from confounding the comparison.
Test information	No holdout rows, predictions, metrics or classifier identities	Preserves the evaluation boundary and prevents optimization against the test set.
Global statistics	No separately computed means, quantiles, correlations or class quotas	Fidelity must arise from the selected rows; the consequence is weaker protection against rare-mode omission.

V. EXPERIMENTAL SETUP

This section describes the data, protocol and measures used to answer RQ1–RQ4, and justifies each choice.

A. Datasets

We use healthcare data because it is the domain where small data is most common and where the motivation for SDG, more training samples without exposing patients, is strongest [1]. We use public datasets so that every result can be reproduced. Table III lists the six classification datasets. Five come from the UCI repository [29]–[33], and the diabetes dataset is the Pima Indians benchmark [34]. They were chosen because they are widely used benchmarks, they are all small in the sense of Definition 1, and together they cover the factors that make small data harder: sample sizes from 195 to 768, 8 to 30 features, two to five classes, a smallest class from 4.3% to 37.3% of the samples, and smallness indices across the strained and small regimes. We keep the original five-class heart-disease target. Missing values (0.1% of the cells in two datasets) are imputed with the mean for numerical and the mode for categorical features.

B. Protocol

Measuring whether synthetic samples help a classifier requires real samples that the generator has never seen, so each dataset is split into a training partition, given to the generator, and a test partition, used only to score classifiers. In small data a single split is unreliable, because the test partition holds only 39 to 154 samples and the result changes with the split. We therefore repeat the split ten times with seeds 43 to 52 (80/20 holdouts) and report means and standard deviations. In each repetition, the generator receives the training partition and produces as many synthetic samples as there are training samples; the LLM sees a 20% sample of the training partition as context. Generation uses temperature 0.3, a value chosen

Algorithm 1 Generation loop of the control layer

Require: table D^{tr}, schema S, rules R, volume N, strategy s, window W

$\mathcal{P} \leftarrow \mathrm{Select}_s(D^{\mathrm{tr}}, \lceil 0.2|D^{\mathrm{tr}}| \rceil)$; $A \leftarrow \emptyset$; $E \leftarrow [\,]$
for chunk $t = 1, \ldots, \lceil N/30 \rceil$ **do**
  $C_t \leftarrow$ rotated slice of $\mathcal{P}$ that fits $0.85W$ ▷ Eq. (4)
  $Y \leftarrow \pi_\phi(S, C_t, \min(30, N - 30(t-1)))$; on overflow shrink C_t by 40%, retry ≤ 3
  $X \leftarrow P_S(Y)$; **if** parse fails **then** $X \leftarrow P_S(\mathrm{repair}(Y))$
  for $j = 1, 2$ **do** ▷ targeted correction
    if $V_{S,R}(X)$ has no violations **then break**
    $X \leftarrow P_S(\pi_\phi(\text{grouped violations of } X))$; append to E
  end for
  $A \leftarrow A \cup X$
end for
$u \leftarrow 0$
while $|A| < N$ **and** fewer than 50 rounds **and** $u < 5$ **do** ▷ Eq. (5)
  $X \leftarrow P_S(\pi_\phi(S, \text{first 20 rows of } \mathcal{P}, \min(30, N - |A|)))$
  if $X = \emptyset$ **then** $u \leftarrow u + 1$ **else** $A \leftarrow A \cup X$; $u \leftarrow 0$
end while
return first N rows of A, together with E, token counts and timings

TABLE III
EVALUATION DATASETS. n: SAMPLES; d: FEATURES; k: CLASSES; $n_{\min}$: SAMPLES IN THE SMALLEST CLASS; $n^\star$: SAMPLE COMPLEXITY BY THE FIRST CRITERION OF RILEY ET AL. [2] (SHRINKAGE 0.9, $d(k-1)$ PARAMETERS, ANTICIPATED R^2_{CS} OF 15% OF ITS MAXIMUM); $\sigma = n^\star / n_{\mathrm{eff}}$ WITH n_{eff} THE TRAINING SIZE ($0.8n$).

Dataset	n	d	k	$n_{\min}$ (%)	$n^\star$	σ
Heart disease	303	13	5	13 (4.3)	10,201	42.1
Breast cancer	569	30	2	212 (37.3)	2,302	5.1
Heart failure	299	12	2	96 (32.1)	946	4.0
Diabetes	768	8	2	268 (34.9)	621	1.0
Liver disease	583	10	2	167 (28.6)	808	1.7
Parkinson's	195	22	2	48 (24.6)	1,851	11.9

For heart disease, $n^\star$ applies the binary criterion to the smallest class versus the rest and is an approximation. Parkinson's contains several recordings per subject, so its true n_{eff} is lower and its σ higher than shown.

in advance to favour adherence to the output format while keeping some diversity.

Classifiers: The utility of synthetic data depends on the learner: interpolated samples can help a nearest-neighbour classifier and harm a boosted ensemble. To avoid conclusions tied to one learner, we train ten classifiers that cover the main model families: linear (logistic regression, LR), kernel (support vector machine, SVM), distance-based (k-nearest neighbours, kNN), probabilistic (Gaussian naive Bayes, NB), a single tree (decision tree, DT), bagging (random forest, RF), boosting (gradient boosting, GB; AdaBoost; XGBoost) and a neural network (multilayer perceptron, MLP). Each classifier is trained in three regimes on the same test partition: real samples only, synthetic samples only, and real plus synthetic samples. XGBoost failed on liver disease in every repetition because its labels are coded $\{1, 2\}$; the code now re-encodes them, but filling this cell would require regenerating every configuration, so we report 59 of 60 classifier–dataset cells.

C. Configurations

We evaluate 13 configurations; only those that completed all six datasets and ten repetitions are analysed. To answer RQ1, seven *control-layer variants* use GPT-4o mini: the reference SYNTHAGENT; three variants that disable a component (type rules, post-processing, and a legacy context-join option); dense and diverse context selection; and a variant in which the LLM chooses context size, selection strategy and temperature from a profile of the dataset. To answer RQ4, four *LLMs* run the reference configuration: GPT-4o mini, GPT-4.1 mini, GPT-4o and GPT-4.1. They form a 2×2 design that crosses model generation (4o versus 4.1) with model size (full versus mini). Using one provider keeps the interface, tokenizer and output mode constant, so differences can be attributed to the model; GPT-4o mini, the cheapest, is the reference so that every variant runs at the lowest cost. The control layer itself works with any OpenAI-compatible endpoint, including locally served open models. Three *established generators* use the same splits: SMOTE [5], which balances every class to the majority count and therefore emits only non-majority samples, and CTGAN and TVAE [6] with the default settings of the reference implementation (300 epochs). In total, 13 configurations give 780 dataset–configuration runs and up to 23,400 classifier fits.

Three control-layer variants turn out to change nothing in this setting. With no user rules, disabling post-processing removes a step that does nothing; the context-join option affects only a legacy fidelity score; and disabling type rules cannot change the result because the reference validator never found a violation with GPT-4o mini. Together with the reference, these variants are four independent runs of the same pipeline. Their spread estimates the variation between repeated runs, which we call the *noise floor*: a difference between two configurations smaller than this spread is not evidence of an effect.

D. Measures

Each requirement of Section I is measured as follows. *R1–R3 (reliability):* completeness, the percentage of non-empty values in the synthetic dataset; the number of parser repairs and type violations; and the cost in tokens and generation time. *R4 (fidelity):* computed on the synthetic samples alone, without the real context, for the numerical features $\mathcal{J}$:

$$F_{\mathrm{JSD}} = 100\Big(1 - \tfrac{1}{|\mathcal{J}|}\textstyle\sum_{j \in \mathcal{J}} D_{\mathrm{JS}}(D_j, G_j)\Big), \qquad F_{\mathrm{corr}} = 100\big(1 - \mathrm{mean}\,|\Sigma_D - \Sigma_G|\big), \tag{6}$$

where D_{JS} is the Jensen–Shannon divergence between real and synthetic marginals and Σ_D, Σ_G are the correlation matrices; both scores are 100 for identical distributions. We also report a classifier two-sample test (C2ST) [26]: a random forest is trained to separate real from synthetic samples, and an accuracy close to 0.5 means they are hard to distinguish. For the established generators, we also report the percentage of *valid* samples, those whose values lie in the observed ranges and categories and are integers where the real feature is.

R5 (privacy): a membership-inference attack scores each real sample by its distance to the nearest synthetic sample and reports the ROC AUC for training versus test samples; 0.5 is chance. *R6 (utility):* for metric $m \in \{\text{accuracy}, \text{F1}\}$, dataset d, classifier c and repetition h,

$$\Delta m_{d,c,h} = m\big(D^{\text{tr}}_{d,h} \cup G_{d,h};\, c\big) - m\big(D^{\text{tr}}_{d,h};\, c\big), \qquad (7)$$

with both models scored on the same test partition, so that the variation between test partitions does not appear as an effect of SDG. We use binary F1 for the binary tasks and class-weighted F1 for heart disease. Binary F1 scores the class coded 1, which is the minority class for heart failure and diabetes and the majority class for liver disease and Parkinson's. Changes are reported in percentage points (pp).

E. Statistical Analysis

Repetitions and classifiers are not independent datasets, so we average them within each dataset and treat the six datasets as matched blocks. Configurations are compared with the Friedman test,

$$\chi^2_F = \frac{12M}{Q(Q+1)} \sum_{a=1}^{Q} \bar{\text{rk}}_a^{\,2} - 3M(Q+1), \qquad (8)$$

where $\bar{\text{rk}}_a$ is the average rank of configuration a over $M = 6$ datasets and Q the number of configurations, followed by two-sided Wilcoxon signed-rank tests of the best-ranked configuration against every other, with Holm correction [35], [36]. Per-cell tests of augmented versus real-only training over the ten repetitions are Holm-corrected within each metric. With six blocks the tests have limited power, so we report effect sizes and the noise floor alongside p-values.

VI. Results

A. RQ1: Reliability of the Control Layer

R1–R3 are met: Every analysed configuration completed all six datasets and ten repetitions. The synthetic datasets have 99.7% of their values filled with GPT-4o mini and 100.0% with the other three LLMs. The control layer also recorded when its components were needed. With GPT-4o mini, GPT-4o and GPT-4.1, no response needed a JSON repair and no sample violated a type rule. GPT-4.1 mini, the fastest LLM, produced on average 7.4 type violations per repetition on breast cancer and one malformed response in ten repetitions on diabetes. These were detected and sent back for correction instead of entering the dataset, and the resulting datasets were complete and gave the best mean accuracy change of all configurations (Table VII). A faster but less disciplined LLM can therefore be used, because its failures are caught. Cost is measured for every run: 72,000 to 81,000 tokens per repetition for the LLM configurations (Table IV).

The noise floor: Table IV compares the control-layer variants. The first four rows are the same pipeline, so their spread is the noise floor: JSD 72.43–72.92, correlation 70.83–72.93, mean accuracy change -1.80 to -1.19 pp, mean F1 change -0.18 to $+0.28$ pp and attack AUC 0.523–0.534.

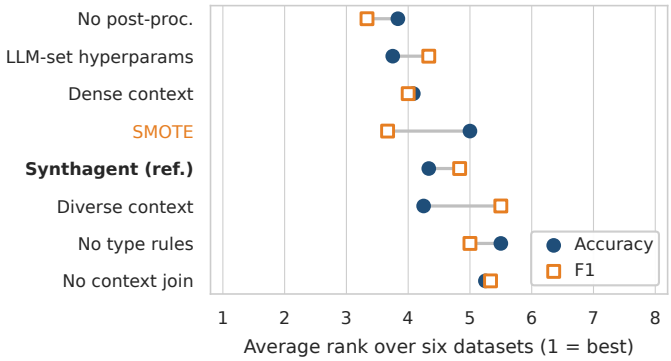


Fig. 3. Average ranks of the control-layer variants and SMOTE over the six datasets (lower is better). Neither accuracy nor F1 shows a significant difference.

Differences of about 0.5 JSD points, 2 correlation points or 0.6 pp of utility between two runs are therefore not evidence of a design effect. Generation time is not a reliable cost measure: the four equivalent runs take 228 to 379 seconds per repetition, depending on the provider, while their token counts differ by less than 2%. We therefore use tokens as the cost measure.

Effect of the design choices: Against this floor, the design choices have small effects. Dense context stays inside the floor on every measure. Diverse context is the only choice that leaves it on fidelity, with the highest JSD (73.27) and correlation (73.72) and the lowest C2ST accuracy (0.872), but also the highest attack AUC (0.544): samples that cover more of the feature space are closer to more real samples. Letting the LLM choose its own settings stays inside the floor on fidelity and utility and costs 57% more tokens (115,245 versus 73,510). The Friedman tests find no difference in utility between the variants ($\chi^2_F(7) = 3.10$, $p = .875$ for accuracy; $\chi^2_F(7) = 4.39$, $p = .734$ for F1), and every Holm-adjusted comparison has $p = 1.0$ (Figure 3).

Answer to RQ1. SYNTHAGENT meets R1–R3 with every LLM analysed: the datasets are complete, failures are detected and corrected, and cost is recorded. Within one LLM, no design choice changes fidelity or utility by more than the noise floor, except a small fidelity gain from diverse context. The value of the control layer is reliability, not a higher score.

B. RQ2: Fidelity and Privacy

Table V reports the reference configuration per dataset. Marginal fidelity averages 72.7 out of 100, from 65.2 on the five-class heart-disease dataset, the smallest in the sense of Definition 1, to 80.2 on diabetes, the largest. The membership-inference attack stays near chance: 0.526 on average and at most 0.567 on any dataset, so the synthetic samples do not reveal which real samples were used. This is a diagnostic for one attack, not a formal guarantee. The C2ST accuracy of 0.878 shows that a classifier can still separate real from synthetic samples; we therefore do not claim that the synthetic samples can replace real ones.

TABLE IV

CONTROL-LAYER VARIANTS (RQ1), ALL WITH GPT-4O MINI. THE FIRST FOUR ROWS RUN THE SAME PIPELINE AND DEFINE THE NOISE FLOOR. SECONDS AND TOKENS ARE MEANS PER REPETITION. BOLD: BEST VALUE AMONG THE LLM CONFIGURATIONS; LOWER IS BETTER FOR C2ST, FOR THE DISTANCE OF THE ATTACK AUC FROM 0.5, AND FOR COST.

	Fidelity (R4) and privacy (R5)				Utility change (R6, pp)		Cost per repetition (R3)	
Configuration	JSD	Corr.	C2ST	Attack AUC	Acc.	F1	Seconds	Tokens
Noise floor: four runs of the same pipeline								
Reference SYNTHAGENT	72.70	72.57	0.878	0.526	−1.19	−0.14	378.7	73,510
No type rules	72.92	72.93	0.876	0.534	−1.80	−0.04	321.6	**72,188**
No post-processing	72.43	70.83	0.883	**0.523**	−1.71	+0.28	251.5	72,235
No context join	72.52	71.18	0.880	0.531	−1.31	−0.18	**228.1**	72,576
Design choices								
Dense context	72.87	70.88	0.878	0.525	−1.24	+0.07	238.8	72,600
Diverse context	**73.27**	**73.72**	**0.872**	0.544	−1.34	−0.13	271.1	73,757
LLM-chosen settings	72.53	70.10	0.886	0.526	−1.25	+0.11	310.2	115,245

TABLE V

REFERENCE CONFIGURATION (GPT-4O MINI) PER DATASET, AVERAGED OVER TEN REPETITIONS AND ALL CLASSIFIERS (RQ2, RQ3). JSD AND CORRELATION ARE ON A 0–100 SCALE.

	R4 and R5				Acc. (%)		Change (pp)	
Dataset	JSD	Corr.	C2ST	AUC	Real	Aug.	Acc.	F1
Breast	77.2	74.5	0.911	0.550	95.1	94.4	−0.66	−0.62
Diabetes	80.2	44.0	0.897	0.504	74.2	74.1	−0.03	+2.39
Heart dis.	65.2	76.6	0.893	0.507	56.6	53.4	−3.15	−0.97
Heart fail.	71.2	81.2	0.892	0.534	74.1	71.6	−2.53	−1.00
Liver	71.8	73.8	0.883	0.495	67.4	67.6	+0.19	+0.13
Parkinson's	70.6	85.3	0.789	0.567	84.9	84.0	−0.97	−0.77
Mean	72.7	72.6	0.878	0.526	75.4	74.2	−1.19	−0.14

Answer to RQ2. The synthetic samples follow the real marginals reasonably well, without reproducing the real samples, but they remain distinguishable from real data.

C. RQ3: Utility for Classifiers Trained on Small Data

Variation of the real-only baselines: Before measuring the effect of synthetic samples, we measure how much classifiers trained on real data alone vary between splits (Table VI). This variation is the yardstick for every utility effect. On the 195-sample Parkinson's dataset, MLP accuracy is $72.3 \pm 18.2\%$, while tree ensembles exceed 92%. On heart failure, SVM and kNN reach 63.3% and 61.3% accuracy but an F1 of 0.0 and 0.7, because they almost never predict the minority class; accuracy alone would hide the failure that SDG is meant to address.

Effect of synthetic samples: With the reference configuration, the effect depends on the dataset and the classifier (Table V, Figure 4). Accuracy improves in 18 of 59 classifier–dataset cells, decreases in 38 and is unchanged in three; F1 improves in 26, decreases in 31 and is unchanged in two. The largest gains occur where real-only training fails on the minority class: kNN on heart failure, with a real-only F1 of 0.7%, gains 15.3 F1 points, and MLP on Parkinson's, the most unstable baseline, gains 8.2 accuracy points. Classifiers that are already strong lose slightly. After Holm correction over 59 tests per metric, no single cell is significant at 0.05. Figure 5 shows the variation between splits: the accuracy change is consistently negative on the two heart datasets, centred on zero for liver disease, and diabetes is the only dataset with a clearly positive F1 change at unchanged accuracy.

Answer to RQ3. With GPT-4o mini, synthetic samples help classifiers that fail on the minority class and slightly harm strong ones; on average they leave F1 unchanged (-0.14 pp) and lower accuracy (-1.19 pp). This is consistent with the conservation result of [1]: synthetic samples can add only the information the generator brings, so large average gains should not be expected. RQ4 asks how much a stronger LLM changes this.

D. RQ4: Comparison with Established Generators and LLMs

Choice of LLM: The LLM is the only factor whose effect exceeds the noise floor (Table VII, Figure 6). All three alternatives to GPT-4o mini exceed it in accuracy change and two in F1 change. GPT-4.1 gives the highest marginal fidelity (75.72, 2.8 points above the floor) and the largest F1 gain ($+1.09$ pp); GPT-4.1 mini gives the best accuracy change (-0.23 pp) at the lowest generation time. The three stronger LLMs preserve pairwise correlations less well (62.8–64.3, against at least 70.83 for GPT-4o mini). Across six datasets, these differences are not significant ($\chi^2_F(3) = 2.00$, $p = .572$ for accuracy; $\chi^2_F(3) = 1.80$, $p = .615$ for F1).

Comparison with established generators: Table VIII compares SYNTHAGENT with GPT-4o mini and GPT-4.1 against SMOTE, CTGAN and TVAE. No generator is best on every measure. CTGAN reaches the highest marginal fidelity (JSD 77.6) but its samples are the easiest to detect (C2ST 0.936), and only 29.6% of them are valid (1.0% on Parkinson's). TVAE keeps 75.6% valid samples and SMOTE 85.7%, but only 14.4% on heart disease, where interpolation produces fractional values in integer-coded features. All five generators keep the membership attack near chance (0.503–0.532). Cost separates them: SMOTE needs 0.015 seconds, TVAE and CTGAN 5.6 and 14.6 seconds of CPU time, and SYNTHAGENT 73,000 to 81,000 tokens and several minutes of API time, with no training.

TABLE VI

REAL-ONLY BASELINES (RQ3) OVER TEN 80/20 SPLITS: MEAN IN PERCENT, STANDARD DEVIATION IN SMALL TYPE. XGBOOST ON LIVER DISEASE (–) FAILED IN EVERY SPLIT BECAUSE THE LABELS ARE CODED $\{1, 2\}$.

	Heart dis.		Breast		Heart fail.		Diabetes		Liver		Parkinson's	
Classifier	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1
RF	61.0 ±4.4	57.0 ±5.9	95.6 ±1.8	95.0 ±2.8	84.3 ±3.3	76.8 ±4.4	74.9 ±2.8	60.4 ±6.0	71.1 ±3.5	81.4 ±2.4	92.3 ±3.2	94.8 ±2.2
DT	53.4 ±3.9	54.0 ±5.0	93.3 ±2.3	92.1 ±4.0	79.2 ±4.7	68.4 ±9.2	71.1 ±3.8	57.2 ±6.4	66.0 ±3.2	76.3 ±3.1	85.1 ±5.6	89.7 ±3.8
MLP	56.6 ±6.4	52.5 ±7.6	94.7 ±0.9	94.0 ±2.1	48.3 ±12.9	34.2 ±24.7	68.4 ±4.1	52.9 ±12.0	68.4 ±4.1	79.4 ±3.6	72.3 ±18.2	77.8 ±26.1
SVM	54.8 ±6.8	39.0 ±7.9	93.3 ±2.1	92.4 ±3.4	63.3 ±5.2	0.0 ±0.0	75.6 ±2.2	57.3 ±4.7	70.7 ±2.9	82.8 ±2.0	81.0 ±8.2	88.5 ±5.2
LR	59.8 ±5.6	56.5 ±5.0	95.2 ±2.0	94.6 ±2.6	80.5 ±3.6	68.9 ±8.4	77.3 ±2.0	62.8 ±5.4	70.3 ±4.0	81.1 ±3.2	86.9 ±5.6	91.3 ±3.9
kNN	55.4 ±6.3	44.4 ±7.9	94.3 ±1.8	93.4 ±3.2	61.3 ±4.1	0.7 ±2.1	75.5 ±1.8	60.0 ±3.3	68.6 ±4.2	79.7 ±3.2	83.6 ±5.6	89.5 ±3.9
NB	55.7 ±5.3	56.9 ±5.0	93.9 ±1.7	93.1 ±3.2	75.0 ±4.9	56.7 ±13.1	76.4 ±2.4	64.0 ±4.9	55.9 ±3.7	55.9 ±3.8	71.5 ±4.4	76.6 ±4.0
GB	54.1 ±5.5	52.8 ±6.3	95.9 ±2.2	95.4 ±2.8	83.0 ±4.9	75.4 ±7.2	74.7 ±3.0	61.7 ±5.8	67.9 ±3.3	78.7 ±2.4	92.1 ±4.0	94.7 ±2.6
AdaBoost	60.8 ±7.6	60.7 ±7.3	97.1 ±1.5	96.8 ±1.7	82.5 ±3.0	74.2 ±6.5	74.9 ±2.2	60.8 ±4.7	67.4 ±3.4	78.2 ±3.3	91.3 ±3.8	94.2 ±2.5
XGBoost	54.3 ±4.6	52.0 ±5.1	97.2 ±1.4	96.9 ±1.8	83.5 ±4.0	75.5 ±7.0	72.7 ±2.8	59.1 ±6.2	–	–	93.3 ±5.2	95.6 ±3.2

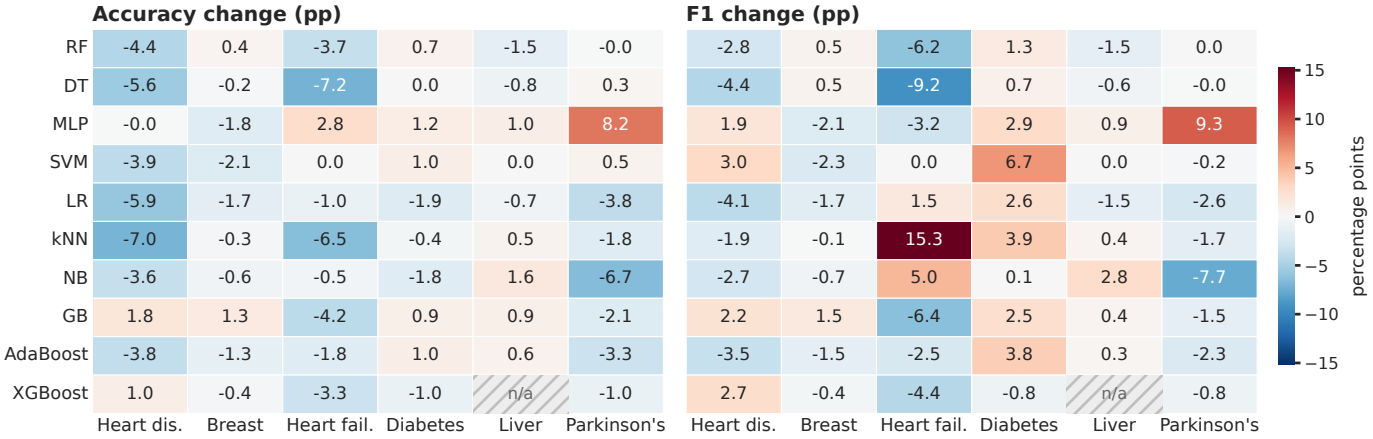


Fig. 4. Change from real-only to real-plus-synthetic training for the reference configuration (RQ3), per classifier and dataset. Hatched cells (n/a) were not evaluated. The same synthetic data can help one classifier and harm another.

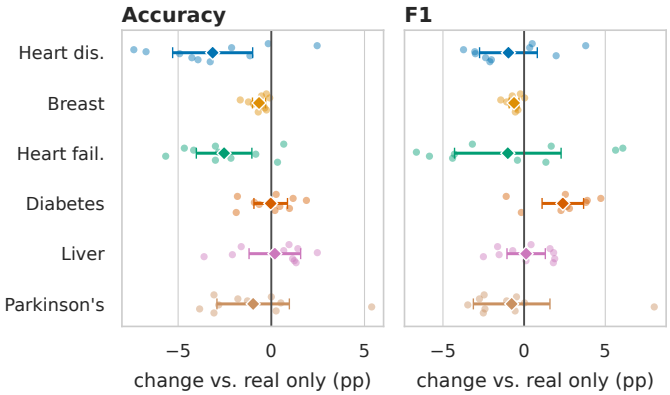


Fig. 5. Variation between splits for the reference configuration (RQ3). Points are splits, after averaging the paired changes over classifiers; diamonds are means and bars 95% t intervals.

Robustness of the utility effect: Average changes hide how much each generator can harm a dataset. Table IX reports the F1 change per dataset. SMOTE gives the largest gains where the scored class is rare (+10.91 pp on heart failure, +4.11 on diabetes) but loses 8.20 pp on liver disease, where the scored class is the majority. CTGAN loses on every dataset,

TABLE VII

LLMS UNDER THE REFERENCE CONFIGURATION (RQ4). SECONDS AND TOKENS ARE MEANS PER REPETITION.

LLM	JSD	Corr.	ΔAcc.	ΔF1	Sec.	Tokens
GPT-4o mini	72.70	**72.57**	−1.19	−0.14	378.7	**73,510**
GPT-4.1 mini	73.60	62.82	−0.23	+0.92	**168.5**	72,069
GPT-4o	73.95	64.32	−0.50	+0.25	255.2	80,046
GPT-4.1	**75.72**	63.30	−0.44	+1.09	201.7	80,652

by up to 10.65 pp. TVAE gives the best accuracy change on average but loses 2.86 pp of F1 on heart failure. SYNTHAGENT with GPT-4.1 has the largest mean F1 gain (+1.09 pp) and the smallest worst case: it never lowers the F1 of a dataset by more than 0.53 pp, while it raises it by 6.35 pp on heart failure. Across datasets, the differences between the five generators are not significant ($\chi^2_F(4) = 6.80$, $p = .147$ for accuracy; $\chi^2_F(4) = 5.60$, $p = .231$ for F1; every Holm-adjusted $p \geq .25$).

Answer to RQ4. The LLM is the main factor in quality and utility. With GPT-4.1, SYNTHAGENT gives the largest and most consistent F1 gain of the five generators, at a higher cost than the established generators and without a significant

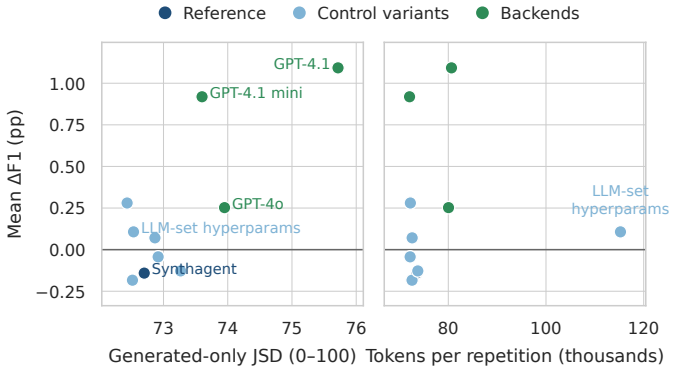


Fig. 6. Fidelity, F1 change and token cost of the LLM configurations (RQ1, RQ4). Control-layer variants cluster inside the noise floor; the LLMs spread along fidelity and F1.

TABLE VIII
GENERATORS COMPARED ON THE SAME SPLITS (RQ4). VALID: % OF SAMPLES WITHIN THE OBSERVED RANGES, CATEGORIES AND INTEGER TYPES (NOT LOGGED FOR THE LLM RUNS, WHOSE TYPES ARE ENFORCED BY THE VALIDATOR). SECONDS: API TIME FOR SYNTHAGENT, CPU TIME OTHERWISE.

Generator	JSD	Corr.	C2ST	AUC	Valid	ΔAcc.	ΔF1	Sec.	Tok.
SYNTHAGENT, 4o mini	72.7	72.6	0.878	0.526	–	−1.19	−0.14	379	73.5k
SYNTHAGENT, 4.1	75.7	63.3	0.877	0.527	–	−0.44	+1.09	202	80.7k
SMOTE†	69.9	85.0	0.795	0.532	**85.7**	−3.08	+0.33	**0.01**	0
CTGAN	**77.6**	75.2	0.936	**0.503**	29.6	−1.40	−3.37	14.6	0
TVAE	71.9	**85.6**	0.868	0.515	75.6	−0.09	−0.21	5.6	0

†Emits only non-majority samples, so its C2ST is not comparable.

advantage across six datasets. TVAE remains a cheap option for accuracy, and SMOTE for a known rare positive class.

VII. DISCUSSION AND LIMITATIONS

What the results show: The objective of this paper was to determine whether a control layer makes LLM-based SDG a reliable and useful option for small data. The answer has two parts. Reliability is achieved: with every LLM analysed, SYNTHAGENT produced complete datasets, caught the failures of the less disciplined LLM and recorded the cost of every run (RQ1), without training on the scarce data. Usefulness is conditional: the synthetic samples are faithful but distinguishable from real ones (RQ2), they help classifiers that fail on the minority class (RQ3), and with a stronger LLM they give the most consistent F1 gain among the generators compared (RQ4), but no generator improves utility significantly on all datasets. This is what the conservation result of our survey predicts [1]: SDG cannot replace missing patients, and its benefit is bounded by the knowledge the generator brings. The LLM is the only generator in our comparison that brings such knowledge, and the control layer is what makes it usable.

Practical recommendations: (1) Choose the LLM first: it is the only factor whose effect exceeds the variation between runs. (2) Measure cost in tokens, not in time. (3) Check the effect of synthetic samples per classifier, on paired splits, before using them. (4) Do not let the LLM choose its own settings without measuring the extra cost.

TABLE IX
MEAN F1 CHANGE PER DATASET (PP) OVER TEN SPLITS AND ALL CLASSIFIERS (RQ4). SA: SYNTHAGENT. WORST: LARGEST LOSS ON ANY DATASET.

Generator	Heart d.	Breast	Heart f.	Diab.	Liver	Park.	Mean	Worst
SA, 4o mini	−0.97	−0.62	−1.00	+2.39	+0.13	−0.77	−0.14	−1.00
SA, 4.1	−0.44	−0.07	+6.35	+1.75	−0.50	−0.53	+1.09	−0.53
SMOTE	−1.50	−0.53	+10.91	+4.11	−8.20	−2.83	+0.33	−8.20
CTGAN	−2.30	−2.00	−10.65	−4.22	−0.65	−0.38	−3.37	−10.65
TVAE	−0.73	−0.88	−2.86	+0.89	+2.21	+0.10	−0.21	−2.86

Deployment across institutions: Because SYNTHAGENT needs only a local dataset, its schema and an inference endpoint, each hospital of a network can run it inside its own infrastructure and decide, with paired local validation, whether to use the synthetic samples before training a model that is later combined by federated learning [37]–[39]. With a remote LLM, the prompt content leaves the institution, so sensitive deployments require a locally served model or a data-processing agreement. We evaluate one site at a time; a multi-site evaluation is future work.

Threats to validity: Only six datasets act as independent blocks, which limits the power of the tests, and the splits are not stratified. The Parkinson's dataset contains several recordings per subject without a subject identifier, so recordings of one person can appear in both partitions. Schema and value ranges are inferred once from the full dataset, which exposes test extremes to the profiler; train-only profiling would be stricter. JSD uses fixed histograms and is dominated by numerical features. The membership attack is a single distance-based adversary, and no result establishes differential privacy [28]. The correction paths were exercised only by GPT-4.1 mini; a stress test with weaker or local models and stricter rules is needed to measure how often each one is needed. In the current implementation, the output of the last correction round and the samples obtained in top-up rounds are not validated again. Context selection is not class-stratified, so a rare class can be missing from a prompt. All LLMs come from one provider, and locally served open models remain to be evaluated; GPT-4.1 nano and o4-mini were started but did not complete every dataset and are excluded. CTGAN and TVAE use default settings, and tuned or diffusion-based generators may do better on some datasets. The smallness indices in Table III rely on a default anticipated R^2_{CS} and are estimates.

Societal impact: Synthetic medical samples can support development, teaching and software testing, but they must not inform diagnosis or policy without domain review. Near-chance performance against one membership attack does not justify releasing sensitive data.

Reproducibility: The implementation, the experiment runner and the script that regenerates every table and figure from the stored results are publicly available.[1]

[1]TODO(authors): add the public repository URL required by the ICDE artifact policy.

VIII. Conclusion

In small data, a generator fitted only to the available samples cannot add information that they lack, while an LLM can bring knowledge from pre-training but produces unreliable output. We presented SYNTHAGENT, a system whose control layer turns LLM responses into complete, validated synthetic datasets of the requested size and records their cost, without training on the target data. On six small medical datasets, SYNTHAGENT met these requirements with every LLM analysed; with GPT-4.1 it gave the largest and most consistent F1 gain of five generators, although no generator was significantly better on all datasets. Repeated runs of the same pipeline vary as much as most design choices, so we recommend that SDG for small data be evaluated against such a noise floor. Future work will evaluate locally served LLMs, class-aware context selection and multi-site deployment.

Acknowledgment

This work is funded by national funds through FCT – Foundation for Science and Technology, I.P., within the scope of the research unit UID/00326 – Centre for Informatics and Systems of the University of Coimbra, https://doi.org/10.54499/UID/00326/2025.

References

[1] S. A. Simões, J. P. Vilela, and P. H. Abreu, "Collectively big, locally small: Formalizing small data in healthcare and learning privately across siloed hospitals," 2026, manuscript under review.

[2] R. D. Riley, K. I. E. Snell, J. Ensor, D. L. Burke, F. E. Harrell, K. G. M. Moons, and G. S. Collins, "Minimum sample size for developing a multivariable prediction model: Part II – binary and time-to-event outcomes," *Statistics in Medicine*, vol. 38, no. 7, pp. 1276–1296, 2019.

[3] V. Borisov, K. Sessler, T. Leemann, M. Pawelczyk, and G. Kasneci, "Language models are realistic tabular data generators," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=cEygmQNOeI

[4] N. Seedat, N. Huynh, B. van Breugel, and M. van der Schaar, "Curated LLM: Synergy of LLMs and data curation for tabular augmentation in low-data regimes," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024.

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[6] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[7] A. Kotelnikov, D. Baranchuk, I. Rubachev, and A. Babenko, "TabDDPM: Modelling tabular data with diffusion models," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 17 564–17 579.

[8] H. Zhang, J. Zhang, Z. Shen, B. Srinivasan, X. Qin, C. Faloutsos, H. Rangwala, and G. Karypis, "Mixed-type tabular data synthesis with score-based diffusion in latent space," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=4Ay23yeuz0

[9] A. V. Solatorio and O. Dupriez, "REaLTabFormer: Generating realistic relational and tabular data using transformers," *arXiv preprint arXiv:2302.02041*, 2023.

[10] J. Kim, T. Kim, and J. Choo, "EPIC: Effective prompting for imbalanced-class data synthesis in tabular data classification via large language models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[11] J. Zhang, G. Cormode, C. M. Procopiuc, D. Srivastava, and X. Xiao, "PrivBayes: Private data release via Bayesian networks," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2014.

[12] C. Lee, J. Kim, and N. Park, "CoDi: Co-evolving contrastive diffusion models for mixed-type tabular synthesis," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 18 940–18 956.

[13] J. Fan, T. Liu, G. Li, J. Chen, Y. Shen, and X. Du, "Relational data synthesis using generative adversarial networks: A design space exploration," *Proceedings of the VLDB Endowment*, vol. 13, no. 11, pp. 1962–1975, 2020.

[14] A. Shankar, H. Brouwer, R. Hai, and L. Y. Chen, "SiloFuse: Cross-silo synthetic data generation with latent tabular diffusion models," in *Proceedings of the IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 110–123.

[15] L. Chen, C. Wu, J. Li, S. Liu, G. Ye, and H. Chai, "TabLoft: Tabular data generation based on LLM with ordered features," in *Proceedings of the IEEE 42nd International Conference on Data Engineering (ICDE)*, 2026, pp. 1044–1056.

[16] J. Hyeong, J. Kim, N. Park, and S. Jajodia, "An empirical study on the membership inference attack against tabular data synthesis models," *arXiv preprint arXiv:2208.08114*, 2022.

[17] C. Ge, S. Mohapatra, X. He, and I. F. Ilyas, "Kamino: Constraint-aware differentially private data synthesis," *Proceedings of the VLDB Endowment*, vol. 14, no. 10, pp. 1886–1899, 2021.

[18] X. Chu, I. F. Ilyas, and P. Papotti, "Holistic data cleaning: Putting violations into context," in *Proceedings of the IEEE 29th International Conference on Data Engineering (ICDE)*, 2013, pp. 458–469.

[19] J. Qin, S. Huang, Y. Wang, J. Zhu, Y. Zhang, Y. Miao, R. Mao, M. Onizuka, and C. Xiao, "BClean: A Bayesian data cleaning system," in *Proceedings of the IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 3407–3420.

[20] A. Narayan, I. Chami, L. Orr, S. Arora, and C. Ré, "Can foundation models wrangle your data?" *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 738–746, 2022.

[21] J. Rorseth, P. Godfrey, L. Golab, D. Srivastava, and J. Szlichta, "Recovering structure in unstructured LLM outputs," in *Proceedings of the IEEE 42nd International Conference on Data Engineering (ICDE)*, 2026, pp. 4329–4330.

[22] P. Li, X. Rao, J. Blase, Y. Zhang, X. Chu, and C. Zhang, "CleanML: A study for evaluating the impact of data cleaning on ML classification tasks," in *Proceedings of the IEEE 37th International Conference on Data Engineering (ICDE)*, 2021, pp. 13–24.

[23] S. Guha, F. A. Khan, J. Stoyanovich, and S. Schelter, "Automated data cleaning can hurt fairness in machine learning-based decision making," in *Proceedings of the IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 3747–3754.

[24] Z. Qian, R. Davis, and M. van der Schaar, "Synthcity: A benchmark framework for diverse use cases of tabular synthetic data," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[25] A. D. Lautrup, T. Hyrup, A. Zimek, and P. Schneider-Kamp, "SynthEval: A framework for detailed utility and privacy evaluation of tabular synthetic data," *Data Mining and Knowledge Discovery*, vol. 39, no. 1, p. 6, 2025.

[26] D. Lopez-Paz and M. Oquab, "Revisiting classifier two-sample tests," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://openreview.net/forum?id=SJkXfE5xx

[27] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *2017 IEEE Symposium on Security and Privacy*, 2017, pp. 3–18.

[28] C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy*, ser. Foundations and Trends in Theoretical Computer Science. Now Publishers, 2014, vol. 9, no. 3–4.

[29] A. Janosi, W. Steinbrunn, M. Pfisterer, and R. Detrano, "Heart disease," UCI Machine Learning Repository, 1988.

[30] W. Wolberg, O. Mangasarian, N. Street, and W. Street, "Breast cancer wisconsin (diagnostic)," UCI Machine Learning Repository, 1993.

[31] "Heart failure clinical records," UCI Machine Learning Repository, 2020.

[32] B. V. Ramana, M. S. P. Babu, and N. B. Venkateswarlu, "ILPD (indian liver patient dataset)," UCI Machine Learning Repository, 2012.

[33] M. Little, "Parkinsons," UCI Machine Learning Repository, 2007.

[34] J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus," in *Proceedings of the Annual Symposium on Computer Application in Medical Care*, 1988, pp. 261–265.

[35] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.

[36] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[37] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Ag"uera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[38] N. Rieke, J. Hancox, W. Li, F. Milletar'i, H. R. Roth, S. Albarqouni, S. Bakas, M. N. Galtier, B. A. Landman, K. Maier-Hein, S. Ourselin, M. Sheller, R. M. Summers, A. Trask, D. Xu, M. Baust, and M. J. Cardoso, "The future of digital health with federated learning," *npj Digital Medicine*, vol. 3, no. 1, p. 119, 2020.

[39] M. J. Sheller, B. Edwards, G. A. Reina, J. Martin, S. Pati, A. Kotrotsou, M. Milchenko, W. Xu, D. Marcus, R. R. Colen, and S. Bakas, "Federated learning in medicine: Facilitating multi-institutional collaborations without sharing patient data," *Scientific Reports*, vol. 10, p. 12598, 2020.